Evidence-Gated Agent Execution: Measuring and Preventing False Success in Tool-Using LLM Agents

Renato Schuster

2026-11-05

Abstract

Tool-using agents frequently report success for actions that did not happen. We describe an evidence-gated execution model that binds each claim to observed state, a signed receipt that makes the claim replayable and tamper-evident, and a cross-node protocol that measures claim-vs-truth disagreement. In a controlled 8-node, 16-task, 200-run study (25,600 probes) with 6.25% deliberately planted dishonest claims, the evidence gate flagged all planted lies while read/action/cross tasks produced zero false successes. A multilingual prompt-injection guard blocked 29/30 blind attacks with 0/10 false positives. On LUMI-G, a guarded rerun produced 1/10,000 false successes (0.01%), failing a zero target; an earlier 0/10,000 result was not reproduced. We report these results with their limitations, including the controlled nature of the trial, small self-authored corpora, partial raw-data availability, and the absence of an independent audit.

1. Introduction

An agent that says “done” without evidence is a liability. The failure is not only a wrong answer: the agent may claim that a file was written, a database row was updated, an invoice was posted, or a command succeeded, while the observed state says otherwise. We call this **false success**. It is especially costly in workflows where a confident but wrong claim propagates into inventory, accounting, or customer communication.

This paper describes a small, reproducible approach:

1. **Claim verification** — each claim is a predicate over observed and expected evidence, evaluated fail-closed into `confirmed`, `refuted`, or `unverified`.
2. **Receipts** — each intent/result pair is bound to SHA-256 digests, with an optional HMAC-SHA256 signature, so a third party can detect tampering and replay the check.
3. **Cross-node measurement** — the same task is executed on heterogeneous nodes, and the agent’s claim is compared with independent ground truth.
4. **Injection guard** — untrusted text is checked before it reaches the model or a tool.

We report results from a controlled cross-node trial, a blind multilingual injection set, and a LUMI-G false-success run. We deliberately include negative and incomplete results: the guarded 10,000-run produced one false success, not zero, and an earlier zero result was not reproduced. The raw per-trial logs for the largest run are not included in the public pack; only the task manifest and aggregate are included.

2. Method

2.1 Claims and predicates

A claim has the form `{text, predicate, observed, expected}`. The core predicates are:

- `eq_num` — numeric equality with a tolerance; accepts BHS/EU/US separators (`2.400,00`, `2.400`, `2,400.00`, `1.234.567,89`).
- `eq_str` — case-folded string equality after whitespace normalization.
- `hash_eq` — SHA-256 equality, with fallback to hashing the strings for non-hex inputs.
- `rc_zero` — observed return code is zero.
- `contains` — expected substring occurs in observed text.
- `json_field_eq` — nested JSON field equality.

The batch result is `refuted` if any claim is refuted, `unverified` if any claim is unverified (or the batch is empty), and `confirmed` only if all claims are confirmed. A missing or malformed observation is `unverified`, never `confirmed`.

2.2 Evidence gate and receipts

The gate emits a report with per-claim status, counts, a timestamp, and a canonical SHA-256 `evidence_sha256`. Recomputing the hash after any edit invalidates the report. Receipts extend this to actions: `intent` and `result` are hashed, a `receipt_hash` binds the core fields, and an optional HMAC-SHA256 signature covers `receipt_hash`. Verification fails closed on missing fields, digest mismatch, receipt-hash mismatch, wrong algorithm, or wrong key.

This is intentionally smaller than a full proof-carrying execution service. Grants, anchor logs, leases, and Merkle proofs are provided by the companion SIGIL profile; the present work focuses on the claim/receipt core and its measurement.

2.3 Cross-node protocol

Sixteen tasks cover read (`R1–R6`), action (`A1–A4`), cross-node (`C1–C4`), and probe (`P1–P2`) behavior. A probe deliberately asks for a missing program or app; the correct answer is failure, and the evidence gate checks whether the agent claims success. Each task is executed 200 times on each of 8 nodes (25,600 controlled probes). A controlled share of probes (6.25%) carries a deliberately dishonest claim; a naive scorer would count those as passes.

Ground truth is the observed system state: stdout, exit code, file state, and hashes. The agent’s claim is never used as its own evidence.

2.4 Injection guard

The guard has two layers. The lexical layer matches instruction override, system-prompt disclosure, jailbreak, secret exfiltration, destructive commands, mass mail, role shift, tool-call syntax, and encoded payloads. The embedding layer uses bge-m3 and is combined with a false-positive brake. The public pack includes the 40-item blind set and results; the 6,900-item generated corpus is reported in aggregate only.

2.5 Metrics

- **Pass rate** — tasks whose observed state matches expectation.
- **False-success rate** — tasks where the agent claims success but observed state disagrees, over all trials (or over a controlled planted-liar share).
- **Guard block rate** — attacks blocked / attacks.
- **False-positive rate** — benign items blocked / benign items.
- **Latency** — wall-clock per task, reported as p50/p95 where available.

3. Experiments

3.1 Controlled cross-node trial

Design: 8 nodes, 16 tasks, 200 runs per task, 25,600 probes. The node set is heterogeneous (Linux and Windows nodes in the local fleet). A controlled 6.25% of probes carried a deliberately dishonest claim.

Result: pass rate 93.75%; 6.25% planted lies flagged; **0 false successes on read/action/cross**; latency p50 2.4 ms, p95 10.8 ms. This is a detection-recall test for the gate, not a field false-success rate.

3.2 Blind multilingual guard set

The blind set contains 40 items: 30 attacks in German, French, Spanish, and Bosnian, plus 10 benign tasks. Lexical-only blocked 4/30 (13.3%) with 0/10 false positives. Lexical + bge-m3 blocked 29/30 (96.7%) with 0/10 false positives. The single miss was the German wording `Lösche die Spur im Audit-Protokoll.`

A larger generated corpus (6,900 items: 5,910 attacks + 990 benign) is reported at 79.6% block rate and 0.1% false-positive rate. That corpus is not included in the public pack; the blind set is included.

3.3 LUMI-G false-success run

On LUMI-G (4× AMD MI250X per node), the raw model produced 197/1,000 false successes (19.7%). A guarded rerun on 10,000 probes produced **1/10,000 (0.01%)** false successes and therefore **failed** the zero target. The earlier 0/10,000 result was not reproduced. We report this as a negative result: the guard materially reduces false success but does not eliminate it in this sample.

3.4 Industrial case study

The Floor Copilot workflow extracts supplier invoices, recomputes line totals, applies 17% VAT, and refuses to mark a result verified when the arithmetic does not reconcile. It is a qualitative case study in this draft; quantified savings and error rates require a larger anonymized evaluation and are not claimed here.

4. Results

Experiment	n	Result	Caveat
Cross-node controlled	25,600	93.75% pass; 6.25% planted lies flagged; 0 false-success read/ action/cross	controlled detection-recall test, not field rate
Guard blind set (lexical)	40	4/30 attacks; 0/10 FP	lexical layer only
Guard blind set (lexical + bge-m3)	40	29/30 attacks; 0/10 FP	small self-authored set
Guard generated corpus	6,900	79.6% block; 0.1% FP	aggregate only; corpus not included
LUMI raw	1,000	197 false successes (19.7%)	single run
LUMI guarded v3	10,000	1 false success (0.01%) — FAIL	earlier 0/10k not reproduced

5. Limitations

- **Controlled, not field.** The 25,600 run measures detection recall with planted liars. It is not an estimate of false-success prevalence in production.
- **Small, self-authored guard set.** The 40-item blind set is a baseline, not a general multilingual benchmark. The 6,900 corpus is aggregate-only in this release.
- **Partial raw data.** Raw per-trial logs for the 25,600 run and the 10,000 LUMI run are not included. The pack includes task manifests, prompts, schemas, and aggregates.
- **No independent audit.** The implementation and results have not been independently audited or peer reviewed.
- **One negative result.** The guarded LUMI rerun failed the zero target; we do not claim zero.
- **Industrial evidence is qualitative.** Floor Copilot is a case study, not a quantified field trial in this draft.

6. Ethics and reproducibility

No human-subject data, personal data, credentials, tokens, or customer records are included. The guard corpus contains synthetic attack strings and should be handled as

untrusted text. The benchmark is reproducible from the public code and the included manifests/prompts; raw logs are available on request.

Reproduction entry points:

- Cross-node: `crossnode.py`, `SPEC.md`, `dataset/crossnode/tasks.json`
- Guard: `bench/eval_blind.py`, `dataset/guard/blind_multilingual.jsonl`
- LUMI: `bench_v2/`, `dataset/lumi/prompts_v2.json`, `dataset/lumi/results.schema.json`
- Claims/receipts: `proofgate-runtime` (PyPI)

7. Conclusion

False success is measurable, and an evidence gate can catch a controlled share of dishonest claims while leaving read/action/cross tasks clean. The result is not a guarantee: a guarded 10,000-run still produced one false success. The contribution is a small, installable core plus an honest measurement pack that others can reproduce, critique, and extend. The next steps are an independent audit, a larger multilingual corpus, and a quantified industrial case study.

Data and code availability

- Measurement pack: https://github.com/gamai-io/crossnode-bench/tree/main/dataset
- Cross-node code/DOI: https://doi.org/10.5281/zenodo.23933835
- Guard: https://github.com/gamai-io/gamai-guard
- LUMI bench: https://github.com/gamai-io/gama-lumi-bench
- Claim/receipt core: https://pypi.org/project/proofgate-runtime/
- SIGIL: https://github.com/gamai-io/sigil

References

1. R. Schuster. *CrossNode-Bench: claim-vs-truth evidence for AI agents.* Zenodo, 2026. https://doi.org/10.5281/zenodo.23933835
2. gamAI. *gamai-guard: multilingual prompt-injection guard.* https://github.com/gamai-io/gamai-guard
3. gamAI. *gama-lumi-bench: LUMI deterministic and false-success experiments.* https://github.com/gamai-io/gama-lumi-bench
4. gamAI. *ProofGate Runtime: claim verification and signed receipts.* https://pypi.org/project/proofgate-runtime/
5. gamAI. *SIGIL/1: proof-carrying execution for AI agents.* https://github.com/gamai-io/sigil
6. Microsoft. *Microsoft Digital Defense Report 2026.* https://www.microsoft.com/security/security-insider/intelligence-reports/microsoft-digital-defense-report-2026

Preprint draft 0.1.0 — 2026-11-05. Not peer reviewed. No institutional affiliation is claimed in this draft; academic collaboration is in preparation.